# Kaggle Competition

## Facial Feature Classification Challenge

**ENGE707 Data Engineering and AI**

Leo Cao, 23211788

Auckland University of Technology

# Table of Contents

**1. Introduction** ..... **3**
1.1 Problem Definition and Objectives ..... 3
1.2 Pipeline Overview ..... 3
**2. Dataset Exploration and Understanding** ..... **4**
2.1 Dataset Structure and Target Classes ..... 4
2.2 Class Distribution and Sample Images ..... 4
2.3 Data Quality Assessment ..... 6
**3. Data Preprocessing and Pipeline Preparation** ..... **7**
3.1 Image Preprocessing and Label Encoding ..... 7
3.2 Validation Split and Data Leakage Prevention ..... 8
3.3 Data Augmentation and Efficient Loading ..... 9
**4. Model Selection and Implementation** ..... **10**
4.1 Model Selection and Justification ..... 10
4.2 Model Architecture and Components ..... 11
4.3 Implementation and Reproducibility ..... 12
**5. Training Strategy and Generalisation** ..... **13**
5.1 Experimental Design and Training Configuration ..... 13
5.2 Learning Curves and Training Behaviour ..... 14
5.3 Overfitting and Underfitting Management ..... 15
**6. Model Evaluation and Kaggle Submission** ..... **16**
6.1 Validation Results and Class-Level Performance ..... 16
6.2 Kaggle Submission and Leaderboard Results ..... 18
6.3 Comparison of Validation and Kaggle Performance ..... 19
**7. Discussion, Limitations, and Future Work** ..... **20**
7.1 Model Comparison and Misclassification Analysis ..... 20
7.2 Strengths and Limitations ..... 21
7.3 Improvement Strategies ..... 22
**8. Conclusion** ..... **22**
**References** ..... **23**
**GitHub** ..... **23**

# 1. Introduction

## 1.1 Problem Definition and Objectives

This project aims to create a deep learning pipeline which is able to classify different facial features of cats and dogs. The main classifications are CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth, DogNose. The task is to be able to identify both the animal as well as these classifications regardless of the variations in image size, lighting and crop coverage.



The main objective is to create a reproducible pipeline, evaluate its performance, and to generate predictions for the ENGE707 Kaggle competition. The competition uses the classification accuracy metric, however this report will also go over individual class performance and prediction errors in order to get a better understanding of the models strengths and limitations.

## 1.2 Pipeline Overview

The pipeline begins with dataset exploration, examining the class distribution, image characteristics as well as data quality. This analysis will be used to create appropriate preprocessing, training and validation sets, and keeping duplicates within the same set.

A simple convolutional neural network (CNN) will be used as the original reference model, but later on we will use a pretrained model through transfer learning. The training and validation performances of the models will be evaluated and compared in order to make further improvements and model selection. The secluded model will then produce predictions for the Kaggle submission, and the results will be further analysed to find limitations and possible improvements.

# 2. Dataset Exploration and Understanding

## 2.1 Dataset Structure and Target Classes

The Kaggle competition supplied a dataset that consisted of 63,660 training images and 29,443 test images. For each of the images, they are each classified into one of the classes; CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth or DogNose. For the training images, they contained their target class meaning we can use it to train our model. However for the test images, they are unlabeled and would require our model to predict the targets for Kaggle.

All of the id's for training images were checked to see if they matched up with their corresponding labels in order to find out whether or not there were any missing values, duplicate ids or unmatched images. Analysis showed that image ids and labels remained consistent, which confirmed that the dataset had complete coverage for the supplied labels. The testing images were kept separate and reserved for only the final prediction.

## 2.2 Class Distribution and Sample Images

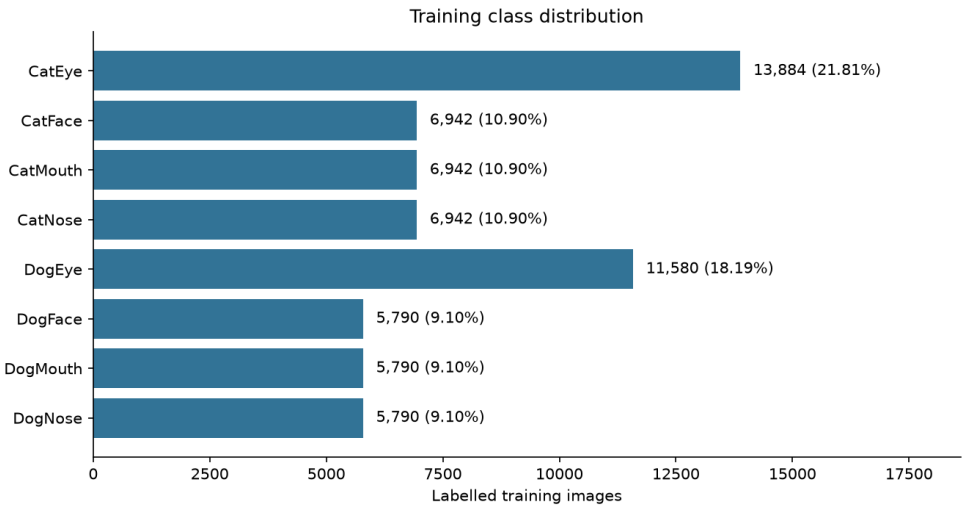



*Figure 1. Distribution of training images across the eight target classes.*

Figure one shows the distribution of the training images across the eight target classes. We can see that the distribution is somewhat unbalanced and being skewed towards 'eye' targets. CatEye is the largest target, making up 21.81% of all the images (13,884 images), while DogFace, DogMouth and DogNose each contain 5,790 images (9.10%). This shows that the largest class is around 2.4 times larger than the smallest class. This shows that overall accuracy will need to be considered alongside class level performance.

Figure 2 shows three randomly selected images for each category in seed 42, and it reveals the complexity of the dataset that the model is supposed to learn. We can see that there are differences in lightning, image size, resolution, and overall quality. Images classified as mouth may also show other features such as nose, this shows that the classification of images is not always a singular isolated feature.




*Figure 2. Three randomly selected training images per class (seed 42).*

## 2.3 Data Quality Assessment

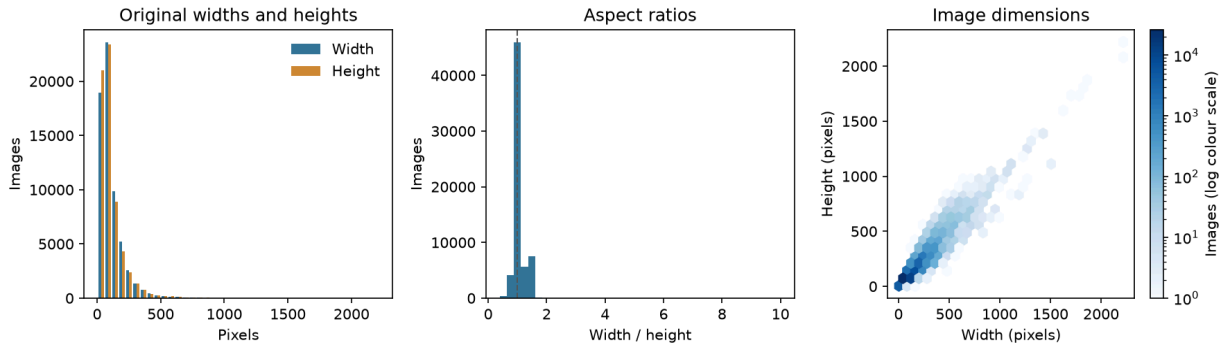



*Figure 3. Training image dimensions and aspect ratios.*

All of the images in the dataset were rgb jpeg and decoded successfully. There was also a huge variation when it came to the dimensions of the image; images sizes ranged from 4 to 2,219 pixels with median widths and heights of 84 and 80 pixels. The graph for aspect ratios alongside image dimensions showed that the majority of the images are relatively small and square. A few factors that would need to be addressed is that very small images compromise on quality, while elongated images may be distorted during resizing.

File hashing and decoded pixel hashing were used and they both identified that there were 15 duplications meaning 30 images were involved, all with consisted labels. Instead of removing these entries, they were instead kept in the same training or validation set in order to prevent any data leakage.

This method of hashing only reveals exact duplicates, and does not reveal images taken at different angles, or images that are just slightly different. This could make it so that these similar images are put in different sets which may compromise the reliability of the validation results.

# 3. Data Preprocessing and Pipeline Preparation

## 3.1 Image Preprocessing and Label Encoding

The images are now preprocessed for the reference CNN model. This first thing done was to resize their longest side to 224 pixels through the use of bilinear interpolation. The original ratios of the images were preserved, and a gray padding was instead added in order to make the images a square ratio of 224 x 244. This method of preprocessing avoided any stretching of elongated images as well as cropping of facial features

Since CNN processes numbers instead of image files directly, the images were converted into rgb float32 tensors with three channels, and the pixel values were normalised to result in values between 0 and 1. Each of the classes were assigned a fixed value from 0-7 based on alphabetical order, and the mapping was saved for later prediction decoding.

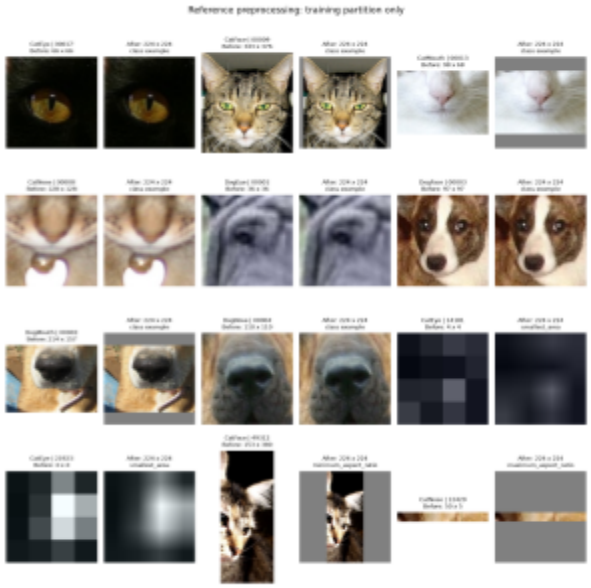


*Figure 4. Training images before and after resizing and centre-padding to 224 x 224 pixels.*

The preprocessing pipeline was tested on 12 training images of different sizes and ratios. This testing confirmed that all the images were consistent with one another, with identical dimensions and pixel values. Figure four visually shows the results of this preprocessing, it also shows that enlarging images that were originally very small does not recover the missing details.

For MobileNetV2, there was an extra normalisation step in preprocessing to adjust the pixel values to match those used when the model was originally trained. Similarly, we kept the same resizing and padding, and these preprocessing steps were applied consistently to the training, validation and test images.

## 3.2 Validation Split and Data Leakage Prevention

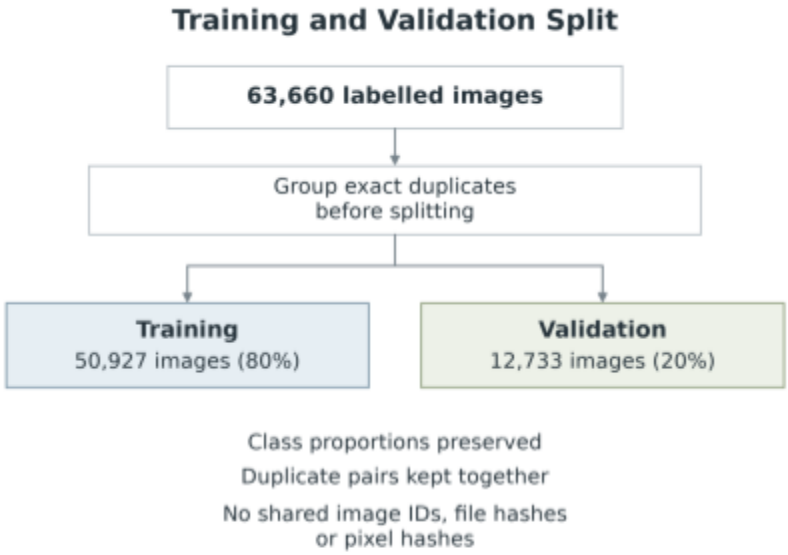



*Figure 5. Class-stratified training and validation split, with exact duplicate groups kept within the same partition.*

Figure 5 summarizes the process on how the data was split into training and validation sets, and how it was done in a way to prevent data leakage. It shows how 63,660 images were split into 50,927 training images (80%) and 12,733 validation images (20%). Before this split was done, images with identical decoded pixels were grouped together. These groups were then split using stratification in order to keep a similar proportion of each class between sets. As mentioned before, duplicate pairs were kept within the same set, and a fixed random seed of 42 was used to make the split reproducible.

All of the images from the dataset have been kept during this process, and both sets contained all 8 classes. Checks were also done to confirm that no duplicate images were shared between sets. These sets were then saved and used for all of the models to make sure training and prediction remained consistent.

## 3.3 Data Augmentation and Efficient Loading

The training images all went through augmentation where they were flipped horizontally randomly, along with small adjustments in contrast and brightness. Figure 6 visually shows us examples of what these changes look like for each class. This method of augmentation introduces more variance to the dataset without accidentally removing important facial features and changing labels. These modifications were applied before the resizing and padding, however the validation images were kept untouched besides the standard preprocessing. The model processes these training images randomly through a batch of 32. The validation images were slightly different where they are kept in the same order

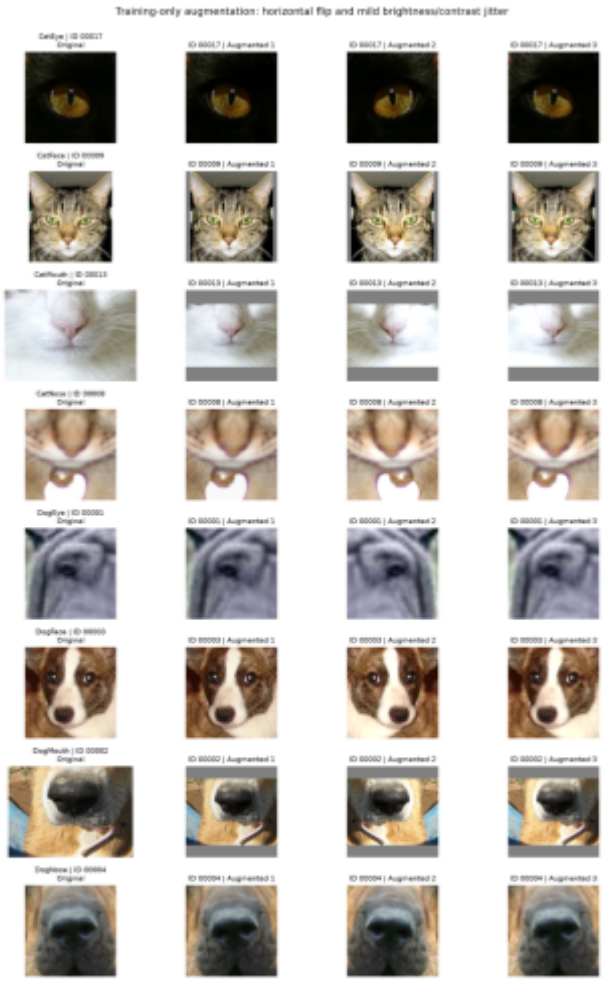



*Figure 6. Examples of training augmentation using horizontal flipping and mild brightness and contrast adjustments.*

A fixed random seed was used to be able to reproduce the data when needed. Further checks were done to check if the images and labels were loaded correctly. Testing was also done to see if training batches could be reproduced using the same seeds.

# 4. Model Selection and Implementation

## 4.1 Model Selection and Justification

This report used three CNN models *(E0: reference CNN, E1: MobileNetV2 with fixed pretrained features, and E2: partially fine tuned MobileNetV2)* and each of the configurations were compared using the same training and validation split. The custom CNN provided a reference for learning directly from the supplied images. This simple structure made it easy to explain the model and allowed for the complete pipeline to be checked before introducing transfer learning.

The second model used was MobileNetV2 which is a pretrained CNN that has already learnt from a large dataset called ImageNet. This model was chosen because it is relatively lightweight and through transfer learning, it is able to be reused to adapt to the eight different classes by replacing its final classifier. This is a much better alternative because it already has existing image features as compared to learning every feature from scratch.

The third model reused MobileNetV2, but this time we used a process called fine tuning where selected feature layers were allowed to learn from the facial images dataset. This was done to see if adapting the pretrained features would be able to improve the overall performance while keeping the earlier layers fixed.

## 4.2 Model Architecture and Components

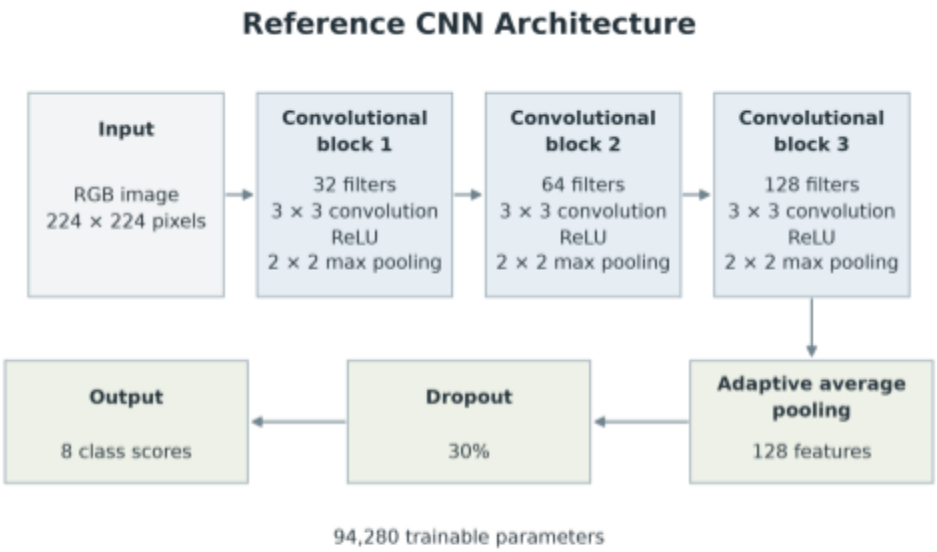



*Figure 7. Reference CNN architecture for eight-class facial feature classification.*

Figure 7 shows the structure of the custom CNN, it takes an input of an rgb 344 x 344 image, and then passes them through three convolutional blocks with 32, 64, and 128 filters. Each of the blocks uses a 3 x 3 convolution, ReLU activation and 2 x 2 max pooling in order to identify visual features as well as reducing the dimensions of the image. The padding preserves the image dimensions during convolution, while pooling reduces the feature map dimension from 112x112 to 56x56 to 28x28. The final features will then be combined using adaptive average pooling which reduces the features to 128 values, followed by a dropout of 0.3 produces the scores for the eight classes. The modal contains 94,280 parameters which were all trained from scratch.

For MobileNetV2, we used a model pretrained on ImageNet and replaced its original classifier to predict the eight classes. During the first experiment, the pretrained feature layers remained unchanged, while only the new classifier was trained. The batch normalisation settings were also kept the same. The model contained 2,234,120 parameters, but only 10,248 of them needed to be trained.

Lastly, for the fine tuning model, we used the previously trained MobileNetV2 model and instead of only training the classifier, we allowed some of its feature layers *(16–18)* to learn alongside the classifier. This increased the number of trainable parameters to 1,205,128. The earlier layers and batch normalisation settings remained unchanged. The aim was to help the model to be able to better recognise the facial features in our dataset while still keeping most of its pretrained knowledge.

## 4.3 Implementation and Reproducibility

The models were developed using PyTorch and Jupyter notebooks in VS Code. The initial testing such as preprocessing were all done locally, however training and predictions of the models were done using "GPU T4 x2" on a Kaggle notebook because local training would take too long.

Once again, all of the experiments were done using the same training and validation splits along with the same class labels to ensure all of the comparisons were unbiased. We also checked that image preprocessing worked consistently across environments. Both the MobileNetV2 experiments used additional normalisation that is needed for their pretrained weights. These training dataset images were copied into Kaggle notebooks in order to improve the overall loading speed, and checks were done to ensure none of the files were changed.

A fixed random seed of 42 was used to improve reproducibility. The fine tuned MobileNetV2 model carried on from the previously trained MobileNetV2 rather than training from scratch. Each experiment's settings, training results, graphs and saved models were recorded separately and stored locally. The saved models were also reloaded and tested to confirm that they produced the same validation results.

# 5. Training Strategy and Generalisation

## 5.1 Experimental Design and Training Configuration

All three of the models used the same 50,927 training images and 12,733 validation images. The same image sizes, class labels and training augmentation were also used, with additional normalisation applied to MobileNetV2. These consistencies in configurations in validation and training images makes it so that only the changes in models and the actual configurations we want observed are isolated,

All the experiments used 'cross entropy loss', the adam optimiser, a batch size of 32 and a fixed random seed of 42. The cross entropy was chosen because each image belongs to exactly one of the eighth classes. Furthermore, we did not add any additional weights to balancing the classes, so additional evaluation should be done to individual class performances.

Adam was used to update the mode's parameters during training, while batches of 32 helped manage memory usage. These settings were chosen as a consistent starting point rather than being tested for the best possible combination.

Both the reference CNN and MobileNetV2 experiments used a learning rate of 0.001, with CNN having 15 epochs and the initial version of MobileNetV2 having 10. The custom CNN was trained entirely from scratch, while MobileNetV2 only trained its new classifier.

For fine tuning, it continued training from the first MobileNetV2 model for an additional 8 epochs. The feature layers had a learning rate of 0.00001 while the classifier used 0.0001. These small learning rates allowed for more gradual adjustments to the already trained model.

The training and validation accuracy and loss were recorded after every epoch. The model with the lowest validation loss was saved. Early stopping was also configured to stop training after three epochs without improvement, and the learning rate could be reduced if there happens to be two epochs without improvement.

## 5.2 Learning Curves and Training Behaviour

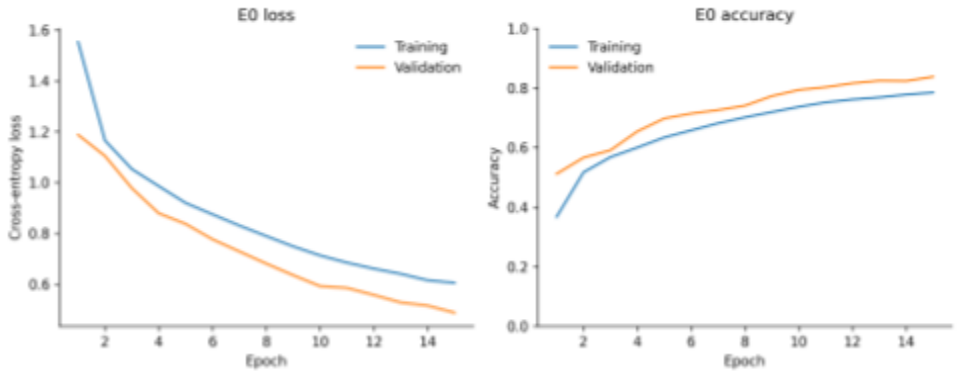



*Figure 8. Training and validation loss and accuracy for the reference CNN.*

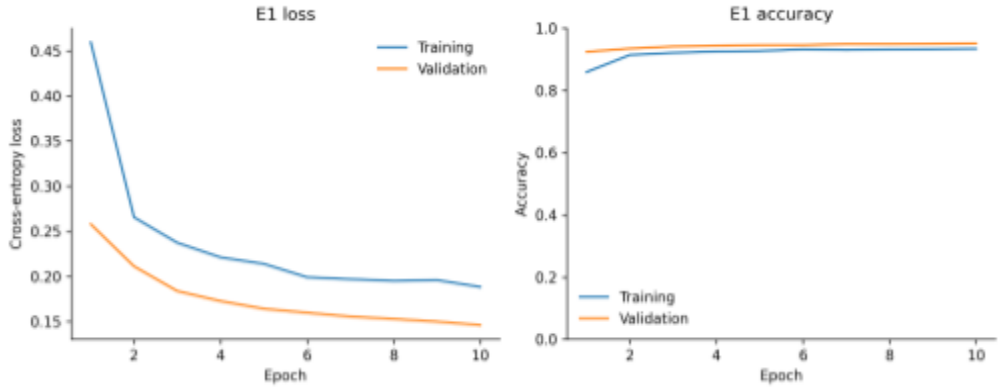



*Figure 9. Training and validation loss and accuracy for MobileNetV2 with only the classifier trained.*

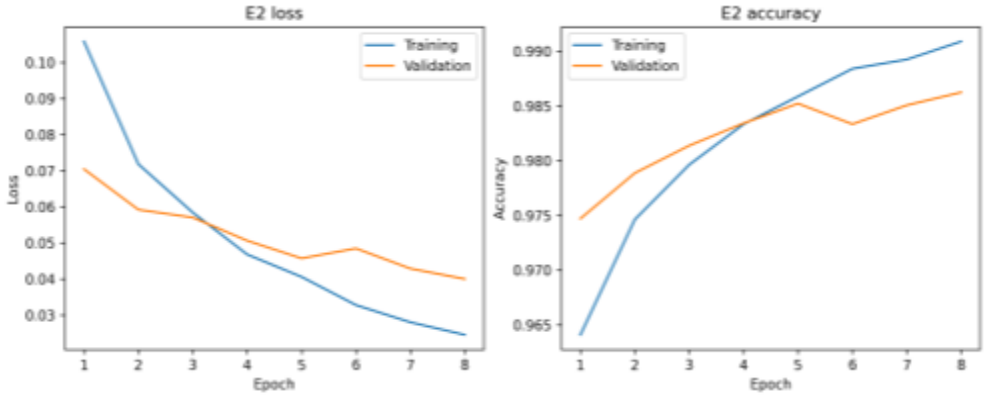



*Figure 10. Training and validation loss and accuracy during MobileNetV2 fine-tuning.*

The figures 8 and 9 show the learning curves for when the reference CNN and initial MobileNetV2 models were training. The validation accuracy for the reference CNN increased from 51.17% to 83.78% over the 15 epochs with a validation loss decreased from 1.1876 to 0.4879. The best result was achieved at epoch 15, and training took around 52.6 minutes.

For the Initial MobileNetV2, the validation accuracy increased from 92.38% to 95.08% over the 10 epochs along with a validation loss decreased from 0.2579 to 0.1461. The best result was achieved at epoch 10, and training took around 39.9 minutes.

Lastly, figure 10 shows the results for the fine tuned MobileNetV2 model. The validation accuracy for the increased from 97.47% to 98.63% over the 8 additional epochs. The validation loss slightly increased at epoch 6 but improved to 0.0400 at epoch 8. The best result was achieved at epoch 8, and training took another 32.2 minutes after the initial MobileNetV2 training.

## 5.3 Overfitting and Underfitting Management

The training augmentation alongside the dropout was previously done to reduce the risk of overfitting which is where the models just memorise training data. This would make it so that the model performs well on the training images but struggles with new images. For the initial MobileNetV2, all of the feature layers were kept fixed, but later on for fine tuning, we only allowed a few selected layers to be able to change, while all the other layers and batch normalisation settings remained fixed.

From the learning curves in section 5.2, both the reference CNN and MobileNetV2 had validation losses that improved after each epoch. This can not be said the same for the fine tuned model where it showed one temporary increase, however it did recover and kept improving afterwards. These are signs that the three models were continuing to improve without any clear signs of overfitting. Furthermore, all of the training for models was not stopped early or had any learning rate reductions, with them all reaching their maximum epoch.

For figure 10, the fine tuned model had a training accuracy of 99.09% and validation accuracy was 98.63% Even though these are very close together, it still does not rule out that over fitting did not occur. This is because these two accuracies were measured under slightly different conditions which means that a direct comparison is less reliable because augmentation and dropout were only used during training.

The validation loss was still improving throughout each epoch, even then it had reached its maximum epoch. This shows that the models may have not finished learning, and that further training may have increased their performance even more, especially for the reference CNN. However this was not tested because the choice of the epochs was purely due to computation time and not to guarantee the best possible results.

# 6. Model Evaluation and Kaggle Submission

## 6.1 Validation Results and Class-Level Performance

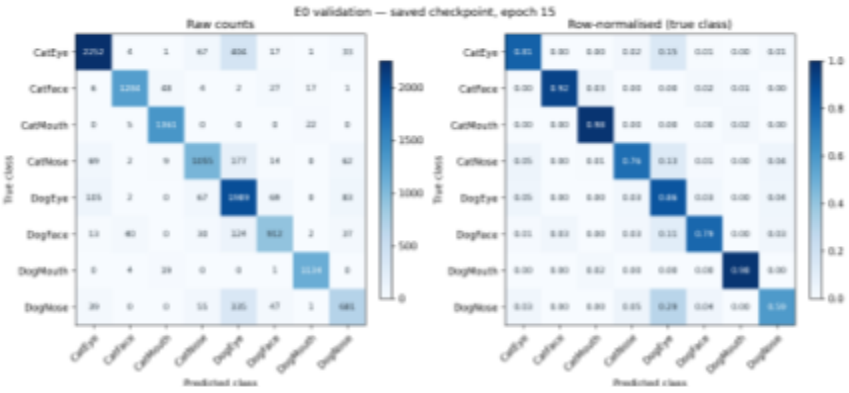



*Figure 11. Validation confusion matrix for the reference CNN.*

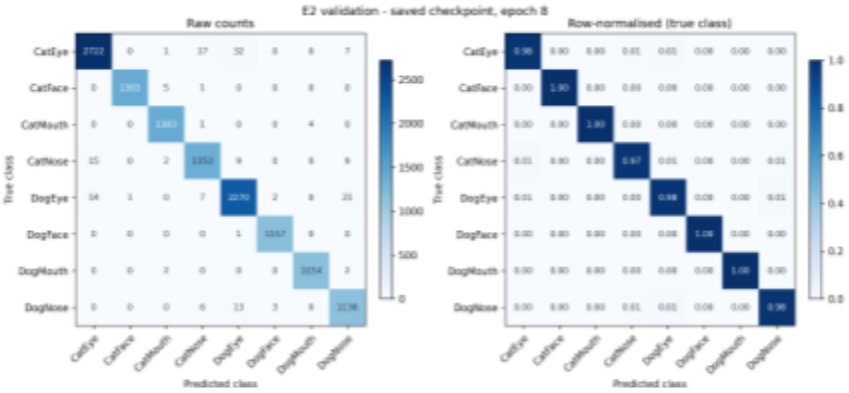



*Figure 12. Validation confusion matrix for fine-tuned MobileNetV2*

All three trained models were tested on the same 12,733 validation images, but this time without augmentation and dropoff. The models were all able to reproduce the exact same validation accuracy and loss. The table below records all of the metrics that were used. Macro F1 equally measures the performance of all classes, while the weighted F1 considers the number of images in each class.

| Model configuration | Accuracy | Cross entropy loss | Macro F1 | Weighted F1 |
|---|---|---|---|---|
| Reference CNN | 83.78% | 0.4879 | 0.8427 | 0.8384 |
| MobileNetV2, classifier training only | 95.08% | 0.1461 | 0.9491 | 0.9509 |
| MobileNetV2, partial fine tuning | 98.63% | 0.0400 | 0.9873 | 0.9863 |

The fine tuned model performed the best out of all three models as it was able to correctly classify 12,558 validation images while only making 175 errors. Before fine tuning, the initial MobileNetV2 model made 626 errors. This increase in performance also increased the validation accuracy by 3.54% along with an improved F1 for all classes. This is why the fine tuned MobileNetV2 model was selected as my final Kaggle submission.

The figures 11 and 12 show the confusion matrices for the reference CNN as well as the fine tuned MobileNetV2. It shows that the class 'CatFace' had the highest F1 score of 0.9975, while DogNose had the lowest of 0.9739. We can also see that the most common mistake made by the fine tuned model was misclassifying 'CatEye' images as 'DogEye' with it classifying 32 CatEye images as DogEye. The following was classifying 21 DogEye images as DogNose. This shows that the model still struggled most on distinguishing between eyes and noses, even though the model achieved high overall accuracy.

## 6.2 Kaggle Submission and Leaderboard Results

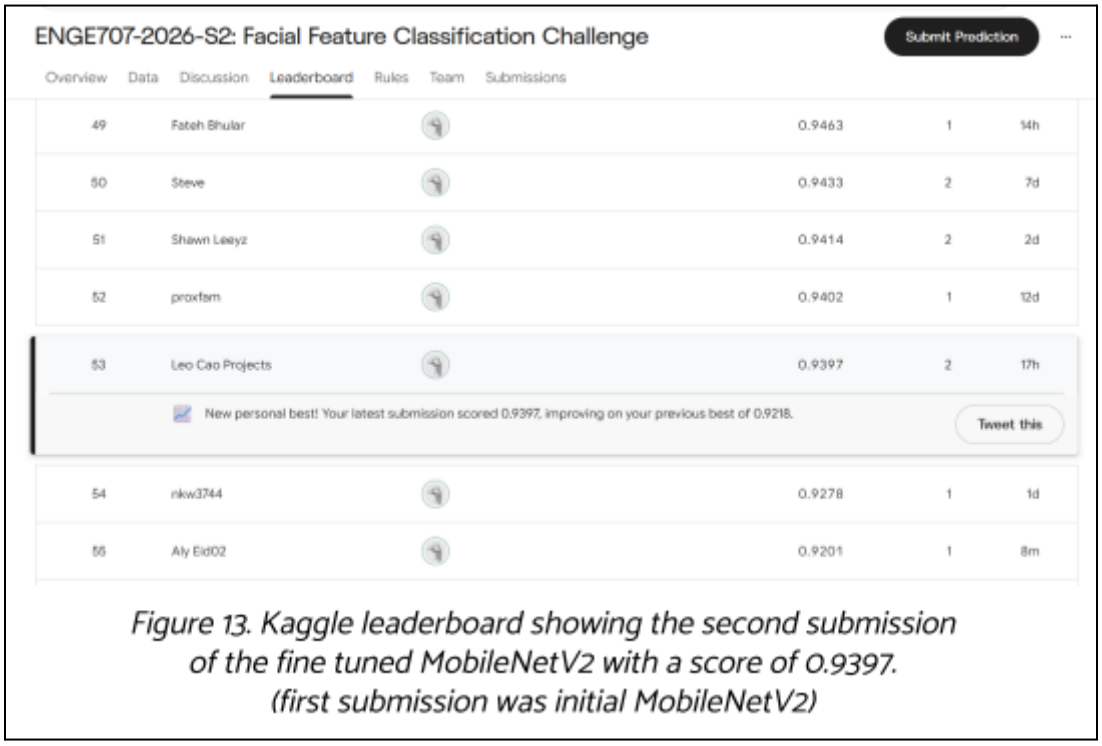



*Figure 13. Kaggle leaderboard showing the second submission of the fine tuned MobileNetV2 with a score of 0.9397. (first submission was initial MobileNetV2)*

Each of the MobileNetV2 models were used to predict each of the 29,443 test images for Kaggle. These test images also went through the same preprocessing and normalisation as the validation images. Each prediction was then matched to their corresponding image ID in the submission order required by Kaggle.

The predictions were saved into a csv file that contained only the original image id alongside its prediction by the models. Testing was performed to ensure that there was complete image coverage, correct class names, and that there were no duplicate entries.

Kaggle accepted both submissions. The original MobileNetV2 model was able to achieve an accuracy of 92.18%, while the fine tuned MobileNetV2 was able to achieve an accuracy of 93.97%. This is an increase of 1.79% which shows that the fine tuned model had an improved performance on the Kaggle test data.

## 6.3 Comparison of Validation and Kaggle Performance

The fine tuned MobileNetV2 model achieves a 98.63% validation accuracy, however it dropped to a 93.97% for the Kaggle accuracy; this is a large difference of 4.66%. Although the fine tuned model did improve both results, the improvement was greater on the validation set *(3.54% increase)* than the Kaggle submission *(1.79% increase)*

The two evaluation sets were not the same and used different images, so the results were not expected to be identical. However, the big difference shows that the validation performance gave a much more optimistic prediction for the hidden score of the Kaggle images. The differences in image difficulty or composition cause explain this reasoning, however without knowing the targets explicitly, the exact reasons cannot be confirmed.

One explanation which is also a limitation is that we used the validation set in order to select the best model of the three, this means that the set was not untouched and the final test was not completely independent. There was also a chance that the images used for validation and training contained similar images in both, which would risk data leakage. Although these factors could have affected our results, we cannot confirm if they did directly cause the difference. The only metric known is that Kaggle suggested our model did not perform as well on the test images, which highlights the importance of why validation accuracy alone is not enough to completely judge the performance of a model.

# 7. Discussion, Limitations, and Future Work

## 7.1 Model Comparison and Misclassification Analysis

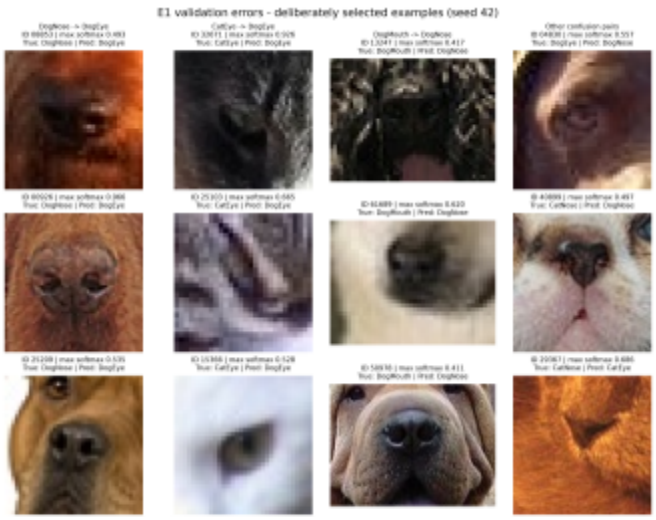



*Figure 14. Selected validation errors from MobileNetV2 before fine-tuning, showing true and predicted classes and maximum softmax scores (seed 42).*

The number of incorrect validation predictions decreased from 2,065 for the reference CNN, to 626 for the initial MobileNetV2, and 175 for the fine tuned MobileNetV2. However the initial MobileNetV2 did not perform better than the CNN for every class; its CatMouth and DogMouth had slightly lower F1 scores. But after fine tuning, the F1 score improved across all eight classes.

Figure 14 shows 12 examples of images that the initial MobileNetV2 had classified incorrectly. Three images were sampled from three of the models most common types of mistakes, along with three other examples, using seed 42. Some of the images were very low quality and had limited detail, while other images contained multiple facial features. For example, some mouth classified images also clearly showed a nose. These characteristics may have made classification harder for the model, however we are unable to determine if this is true.

Some of the incorrect predictions also had extremely high confidence, for example in diagram 12, the image #00926 is a true classification of a 'DogNose ' however the model predicts it as 'DogEye' with a maximum softmax *(confidence)* score of 0.96. This just goes to show how a model can be highly confident while still making an incorrect prediction.

Another thing to mention is that the fine tuning did improve both the performance for validation and Kaggle. However it trained the previous model for additional epochs with a smaller learning rate. This means it is not reasonable to conclude that allowing more feature layers to learn was the only reason for the improvement, and the extra epochs may be a contributing factor.

## 7.2 Strengths and Limitations

One strength of the pipeline approach was that we used the same training and validation sets for all three models which allowed their performance to be compared unbiasedly. Furthermore, we also kept duplicate images in the same set in order to reduce the risk of any data leakage. All trained models were saved and tested again to confirm their results. Alongside the accuracy, we also analysed individual class performance, classification mistakes and Kaggle scores to get a better understanding of how well the models performed.

However, some images were small and contained very little detail. Although resizing them made them suitable for the models, this does not enhance the image or bring back any information that was missing. Furthermore, padding preserved the crop, but it left large borders around images with unusual ratios.

The validation process also had its limitations. Similar but not duplicate images of the same animal may have been distributed between the training and validation sets, resulting in a risk of data leakage. We also only used one dataset split and random seed, and we used validation results to select our best models; meaning the final Kaggle test was not completely independent.

The final model also achieved a validation accuracy of 98.63% compared to the 93.97% on Kaggle, showing that its performance on different images was lower. Therefore, our validation accuracy should not be treated as a guaranteed prediction for future performance. We also did not independently test the individual effects of our preprocessing choices, augmentation or dropout.

### 7.3 Improvement Strategies

One possible improvement would be to make the validation process more reliable. The way this could be done is to identify images from the same animal, we could then keep them in the same training or validation set to further reduce data leakage. We could also repeat our experiments using different random seeds to check whether the results remained consistent.

Future experiments could also test different learning rates and different numbers of layers during MobileNetV2 fine tuning. Their performance could then be compared using validation accuracy, loss and individual class results, rather than relying only on Kaggle scores.

We could also compare models trained with and without augmentation or dropout to understand how much these techniques have actually helped. Furthermore, analyzing images that were incorrectly classified, such as eye and nose images, could help us identify further improvements.

## 8. Conclusion

This project developed an image classification model to classify either different cat and dog facial features. It explored the dataset and identified differences in class sizes, image dimensions and duplicate images. Images were then prepared for training, and the dataset was divided into training and validation sets while keeping identical images in the same set.

The reference CNN achieved a validation accuracy of 83.78%, which increased to 95.08% using MobileNetV2, and fine tuning the MobileNetV2 was able to bring this accuracy up to 98.63%. The fine tuning was able to increase the performance consistently across all eight classes compared to the original MobileNetV2 model.

Both the MobileNetV2 models were submitted to Kaggle. The initial model achieved an accuracy of 92.18%, while fine tuning improved it to 93.97%. However the final Kaggle accuracy was 4.66% lower than the validation accuracy, which shows that the validation performance gave a much more optimistic prediction for the hidden score of the Kaggle images.

Some limitations of our models included images being too small, multiple facial features being in one image, and the possibility that related images appear in both the training and validation sets. Future improvements should be to focus on improving the validation process and testing different training settings to investigate further improvements.

## References


ABSiddique. (2026). *ENGE707-2026-S2: Facial feature classification challenge* [Kaggle competition]. https://www.kaggle.com/competitions/enge-707-2026-s-2-cat-and-dog-facial-feature-classification-ch

*Convolutional neural networks* [ENGE707 Lecture 11 slides]. (2026). Auckland University of Technology.

PyTorch contributors. (n.d.-a). *mobilenet_v2*. Torchvision documentation. https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.mobilenet_v2.html

PyTorch contributors. (n.d.-b). *PyTorch documentation*. https://docs.pytorch.org/docs/stable/


## GitHub

https://github.com/LeoCaoProjects/Kaggle-Competition